App Development Project Report

1. Student Details

- **Name:** SUMIT KUMAR JENA
- **Roll Number:** 23f2002871
- **Email:** 23f2002871@ds.study.iitm.ac.in

- **About Me:** I am a student pursuing a BS in Data Science and Applications at IIT Madras. I have a little interest in backend architecture, full-stack web development, and building data-centric platforms that solve real-world organizational challenges.

2. Project Details

- **Project Title:** Campus Placement Portal Application
- **Problem Statement:** To design and build a centralized web platform that bridges the communication and logistical gap between students, recruiting companies, and the university placement cell, allowing for seamless job postings, application tracking, files management, and automated alerts.
- **Approach:** The application was built as a Single Page Application (SPA) using Vue.js for a dynamic frontend and Flask for a secure, modular backend. It features Role-Based Access Control (RBAC), Redis caching for high-performance data retrieval, and Celery for asynchronous background tasks such as generating CSV reports and scheduled email reminders.

3. AI/LLM Declaration

I used AI tools (such as ChatGPT/ClaudeAI) to assist with debugging complex SQLAlchemy database relationships, understanding the integration between Celery and Redis for asynchronous background tasks, and learning Vue.js reactive rendering concepts. The extent of AI usage is around 20-25%, limited to conceptual explanations and code snippet suggestions. All implementation logic, architectural structuring, routing, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend Python web framework
Vue.js (CDN)	Frontend framework for dynamic UI and reactivity
SQLAlchemy	Object Relational Mapper (ORM) for the database

Technology / Library	Purpose
SQLite	Lightweight local database for storing application data
Redis	In-memory datastore used for API caching and as a message broker
Celery & Celery Beat	Asynchronous task queue for background jobs and cron-scheduled tasks
Flask-JWT-Extended	Secure user authentication and role authorization
Bootstrap 5	Frontend styling and responsive grid design
Axios	Handling REST API HTTP requests from the frontend to the backend
MailHog & Flask-Mail	Local SMTP server testing and email generation

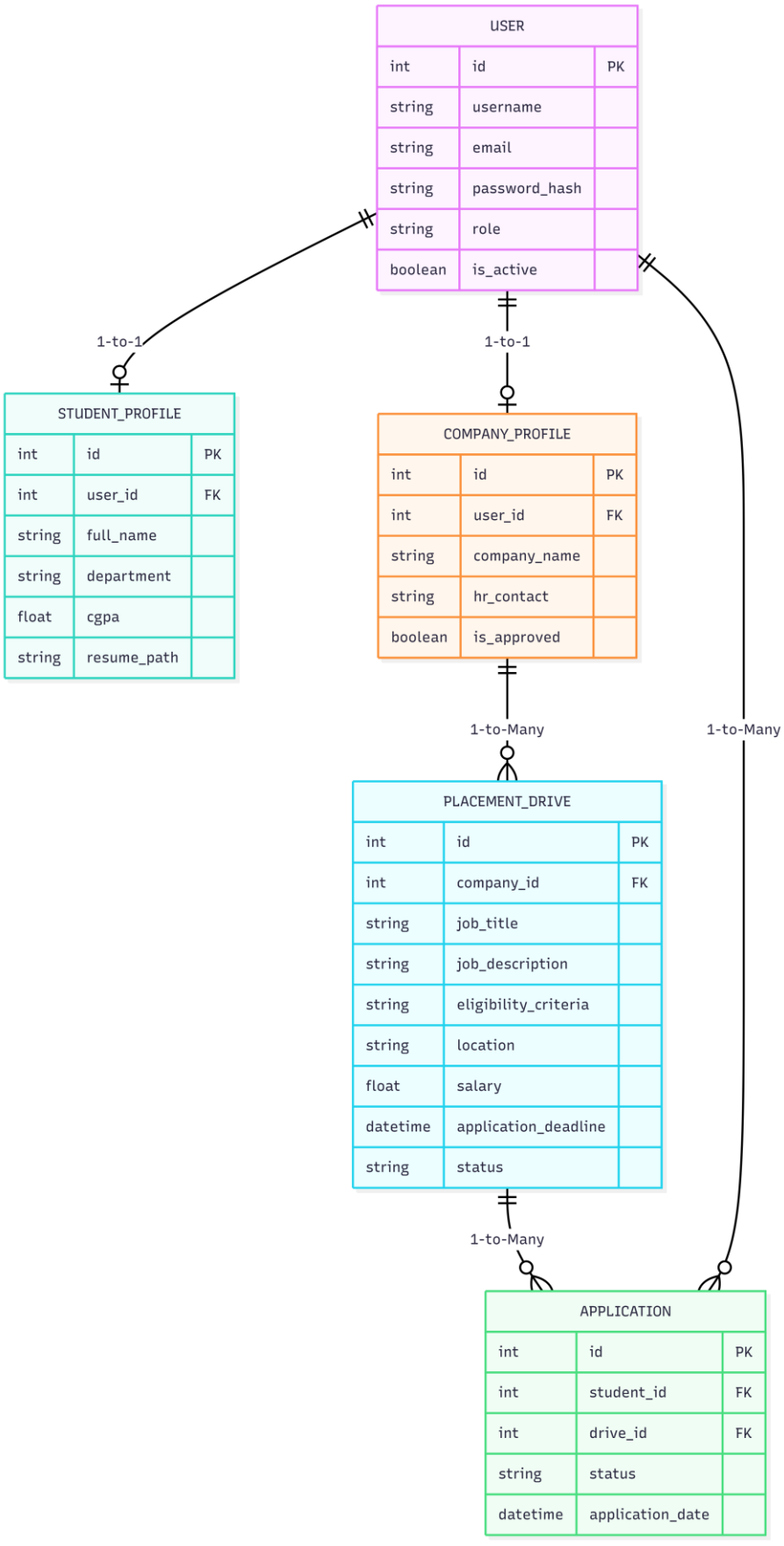
5. Database Schema / ER Diagram

Tables:

1. **User** - stores account login details (id, username, email, password_hash, role, is_active)
2. **StudentProfile** - stores academic details (id, user_id, full_name, department, cgpa, resume_path)
3. **CompanyProfile** - stores organization details (id, user_id, company_name, hr_contact, is_approved)
4. **PlacementDrive** - stores job postings (id, company_id, job_title, job_description, salary, application_deadline)
5. **Application** - logs student job applications (id, student_id, drive_id, status, application_date)

Relationships:

- **One-to-One:** User → StudentProfile / CompanyProfile
- **One-to-Many:** CompanyProfile → PlacementDrive
- **One-to-Many:** PlacementDrive → Application
- **One-to-Many:** User (Student) → Application


USER
int id PK
string username
string email
string password_hash
string role
boolean is_active
1-to-1
1-to-1
STUDENT_PROFILE
int id PK
int user_id FK
string full_name
string department
float cgpa
string resume_path
COMPANY_PROFILE
int id PK
int user_id FK
string company_name
string hr_contact
boolean is_approved
1-to-Many
1-to-Many
PLACEMENT_DRIVE
int id PK
int company_id FK
string job_title
string job_description
string eligibility_criteria
string location
float salary
datetime application_deadline
string status
1-to-Many
APPLICATION
int id PK
int student_id FK
int drive_id FK
string status
datetime application_date

6. API Resource Endpoints

Endpoint	Method	Description
`/api/auth/login`	POST	Authenticate user and generate JWT session token
`/api/auth/register/<role>`	POST	Register new student or company account
`/api/admin/pending-companies`	GET	Fetch companies waiting for admin approval
`/api/company/drives`	GET/POST	Fetch existing company jobs or create a new job posting
`/api/student/profile`	GET/POST	Fetch or update student academic details and upload PDF resume
`/api/student/apply/<drive_id>`	POST	Submit a job application for a specific drive
`/api/student/export-csv`	POST	Trigger Celery background task to asynchronously export history

7. Architecture and Features

Architecture Overview:

- `app.py` - main Flask application entry point
- `/models.py` - database schema definitions using SQLAlchemy
- `/extensions.py` - Initialization of external libraries (db, cache, mail)
- `/routes/` - Flask Blueprints separating logic by user role (auth, admin, student, company)
- `/tasks.py` - Celery background workers and scheduled cron jobs
- `index.html` - Complete Vue.js frontend

Implemented Features:

- Secure JWT Authentication with Role-Based Access Control (Admin, Company, Student).
- Student academic profile management and secure PDF resume file uploads.
- Live search and filtering on all user dashboards using Vue.js computed properties.
- API Caching implemented via Redis to speed up database-heavy retrieval routes.
- Asynchronous CSV data exports handled by Celery background workers to prevent UI blocking.
- Automated daily reminder emails and monthly activity reports sent via Celery Beat scheduling.

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1pzPPgNsmMBExu9GpnbH3fm5gNxJjeGLN/view?usp=drive_link